How-To Library

Informatica

Enable SSL Authentication in Apache Kafka using Portecle

© Copyright Informatica LLC 2020, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Abstract

Apache Kafka clients and brokers can communicate over SSL using a dedicated port. However, to use SSL authentication in Apache Kafka you must enable it. This article describes the steps to enable SSL authentication in Apache Kafka using Portecle.

Supported Versions

- Data Engineering Streaming 10.4.0

Table of Contents

Overview. . . . 2
Prerequisites. . . . 2
Creating Keystores and Truststores using Portecle 2
Step 1. Create Keystore Files 3
Step 2. Create Truststore Files 4
Enable SSL Authentication on a Kafka Node. . . . 5
Copying the Truststore File to Nodes in the Kafka Cluster 6
Create an SSL Enabled Kafka Connection in Informatica Domain. . . . 7

Overview

Apache Kafka is a publish-subscribe messaging system that manages and maintains the real-time stream of data. With Kafka, any user or application can write any messages to any topic or read data from any topic.

To communicate securely with Kafka, you can use SSL authentication. Using SSL authentication, you can encrypt any communication between Kafka brokers, or between client and broker.

Portecle is a user-friendly GUI application that you can use to create, manage, and examine keystores, keys, certificates, certificate requests, and certificate revocation lists.

Prerequisites

Before you create an SSL enabled Kafka connection, perform the following tasks:

- Install the latest version of Apache Kafka. For more information, see https://kafka.apache.org/downloads.
- On Windows, install Portecle. For more information, see http://portecle.sourceforge.net/.
- For UNIX or Linux, download the certificate generation script from https://github.com/confluentinc/confluent-platform-security-tools/blob/master/kafka-generate-ssl.sh.

Follow the on-screen instructions to complete the installation.

Creating Keystores and Truststores using Portecle

When you enable secure communication for Apache Kafka, Informatica requires keystores and truststores. Create the keystore and truststore files and then configure a certificate for higher level of security.

Step 1. Create Keystore Files

Create and use a keystore to store the private key and own identity certificate required for verification.

1. Open a command prompt as administrator.
2. Enter the following command to start Portecle:

   ```
   java -jar portecle.jar
   ```
3. From the File menu, choose **New Keystore**.

   Alternatively, click the **New Keystore** toolbar button.

   The **New Keystore Type** dialog box appears.
4. Select the new keystore type as **JKS** and click **OK**.
5. Go to **Tools**.

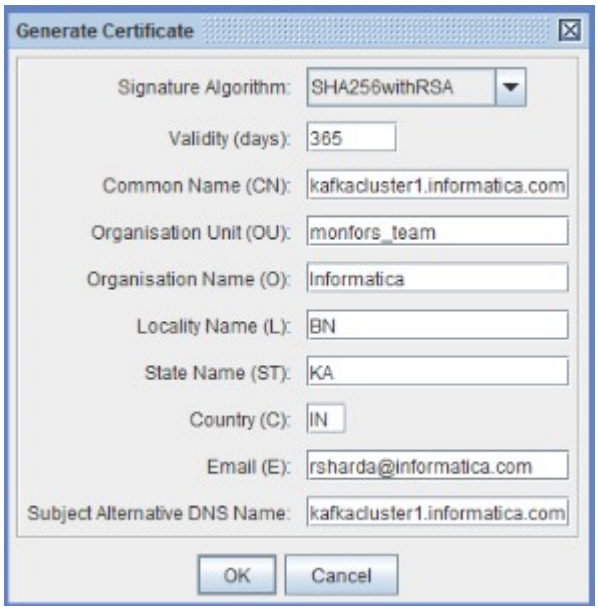
   The title bar changes to display the text **Untitled** and the status bar changes to display the chosen keystore type, **JKS**.
6. From the Tools menu, select **Generate Key Pair**.

   The **Generate Key Pair** window appears.
7. Select **Key Algorithm** as RSA, enter **Key Size** as 2048, and then click **OK**.

   The **Generate Certificate** dialog box appears.
8. Enter the **Common Name (CN)** and the **Subject Alternative DNS Name** with a fully qualified domain name of the Kafka node.

   You need a fully qualified domain name of the Kafka node for complete HTTPS transport encryption with Kafka.

   The following image shows the **Generate Certificate** dialog box and entries required for complete HTTPS transport encryption with Kafka:

9. Enter the other details and then click **OK**.

   The **Key Pair Entry Alias** dialog box appears.
10. Enter an alias or retain the default one and then click **OK**.

    The **Key Pair Entry Password** dialog box appears.

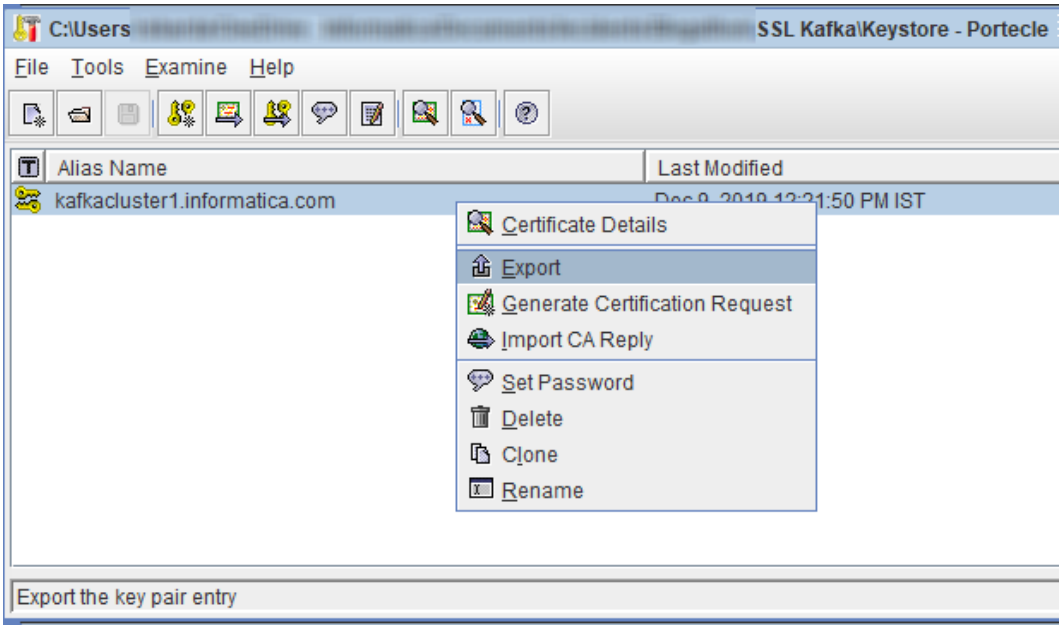
11. Enter and confirm the password and then click **OK**.
12. Press `CTRL+S` to store the keystore file. In the **Set Keystore Password** dialog box, enter same password and then click **OK**.

Step 2. Create Truststore Files

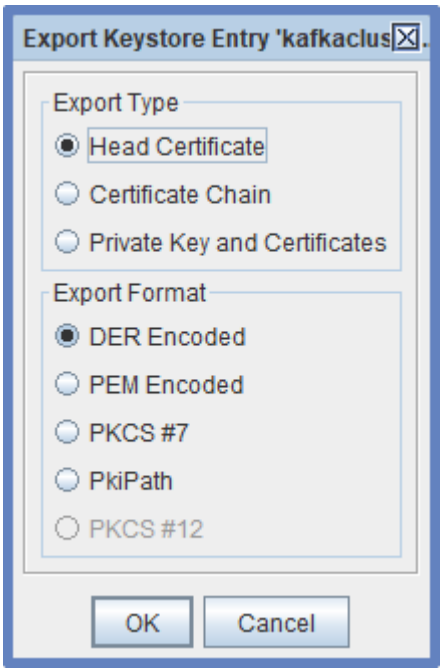
Create a truststore file to store certificates from the trusted Certificate Authority (CA). Use the truststore file to verify the certificate provided by the server in an SSL connection.

Perform the following steps to create a truststore file that contains the Head certificate:

1. Open the keystore file, right-click the file alias name, and select **Export**.



2. Retain the default values and click **OK**.



3. Save the `.cer` file.
4. From the Tools menu, click **Import Trusted Certificate**.

   The **Import Trusted Certificate** dialog box appears.
5. Select the `.cer` certificate file that you exported in Step 3.
6. Confirm the certificate details and click **OK**.

   A dialog box appears asking if you want to accept the certificate.
7. Click **Yes** to trust the certificate and import it.

   Otherwise, click **No** to cancel the file import.

   The **Trusted Certificate Alias** dialog box appears.
8. Enter the alias of the new trusted certificate and click **OK**.

   The new trusted certificate entry appears in the keystore entries table with the chosen alias name.

   Note: In case of multi-broker Kafka, all keypairs for each broker must reside in the same keystore file. Similarly, all head certificates must reside in the same truststore file.

Enable SSL Authentication on a Kafka Node

After you import the certificates to the truststore file, enable the SSL authentication on a Kafka node.

1. Log in to the virtual machine that contains the Kafka installation.
2. Copy the keystore and truststore files to the same machine.
3. Open the `Server.properties` file from the `<Kafka_Home/config/>` path.

4. Edit the following properties:

Properties	Values
Listeners	`SSL://<Fully qualified domain name of Kafka Server>:<port>` For example, `SSL://kafkacluster1.informatica.com:9094`
Advertised Listeners	`SSL://<Fully qualified domain name of Kafka Server>:<port>` For example, `SSL://kafkacluster1.informatica.com:9094`
Security.interbroker.protocol	SSL
ssl.client.auth	Required. This property ensures that the SSL client is always authenticated and does not allow any plaintext connections to Kafka.
ssl.key.password	Key password. This is the same password that is prompted before saving the keystore file.
ssl.keystore.location	Keystore location.
ssl.keystore.password	Keystore password.
ssl.truststore.location	Truststore location.
ssl.truststore.password	Truststore password.
ssl.enabled.protocols	TLSv1.2 or any other protocols
ssl.endpoint.identification.algorithm	HTTPS **Note:** Set the correct fully qualified domain name (FQDN) in the common name and subject alternative name of the certificate. The identification algorithm works correctly only if FQDN is correct in common name and subject alternative name of the certificate. If the FQDN is wrong or any other information is missing, you cannot connect to the Kafka endpoint. In such a case, setting it blank or null resolves your issue. You must be aware that you can be prone to endpoint spoofing attacks.

Copying the Truststore File to Nodes in the Kafka Cluster

For an external cluster deployment, copy the file to the truststore location specified in the Hadoop cluster configuration.

1. Copy the keystore and truststore files to each node in the Kafka cluster.

   The file path must be same on all the nodes of the cluster. For example, `/etc/security/certificates`.

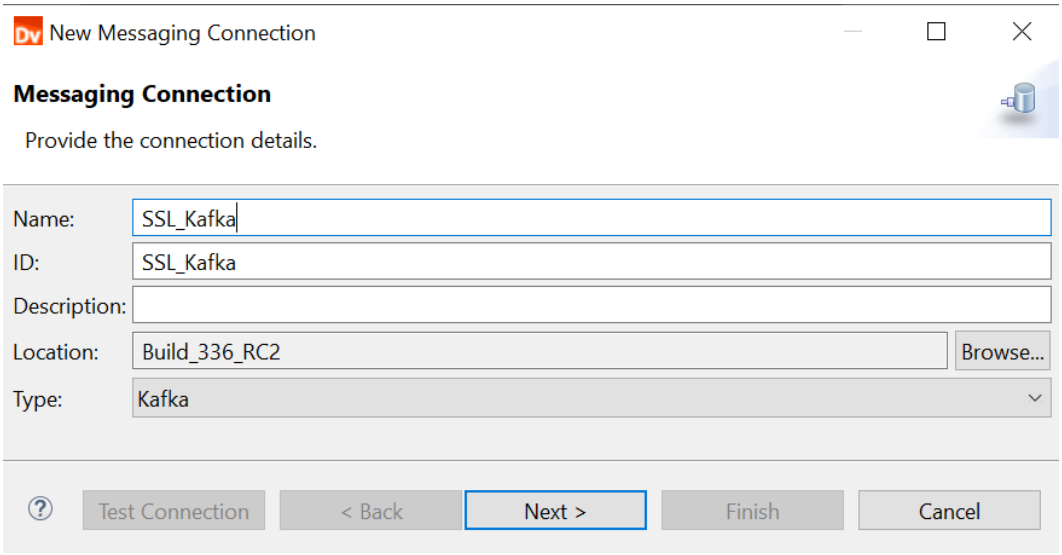
2. Create a similar folder in the machine that hosts the Informatica Developer and copy the certificates to the folder.

Create an SSL Enabled Kafka Connection in Informatica Domain

You can create an SSL enabled Kafka connection in Informatica Developer or Administrator tool.

1. Create a messaging connection. Add the name of the connection and the connection details.

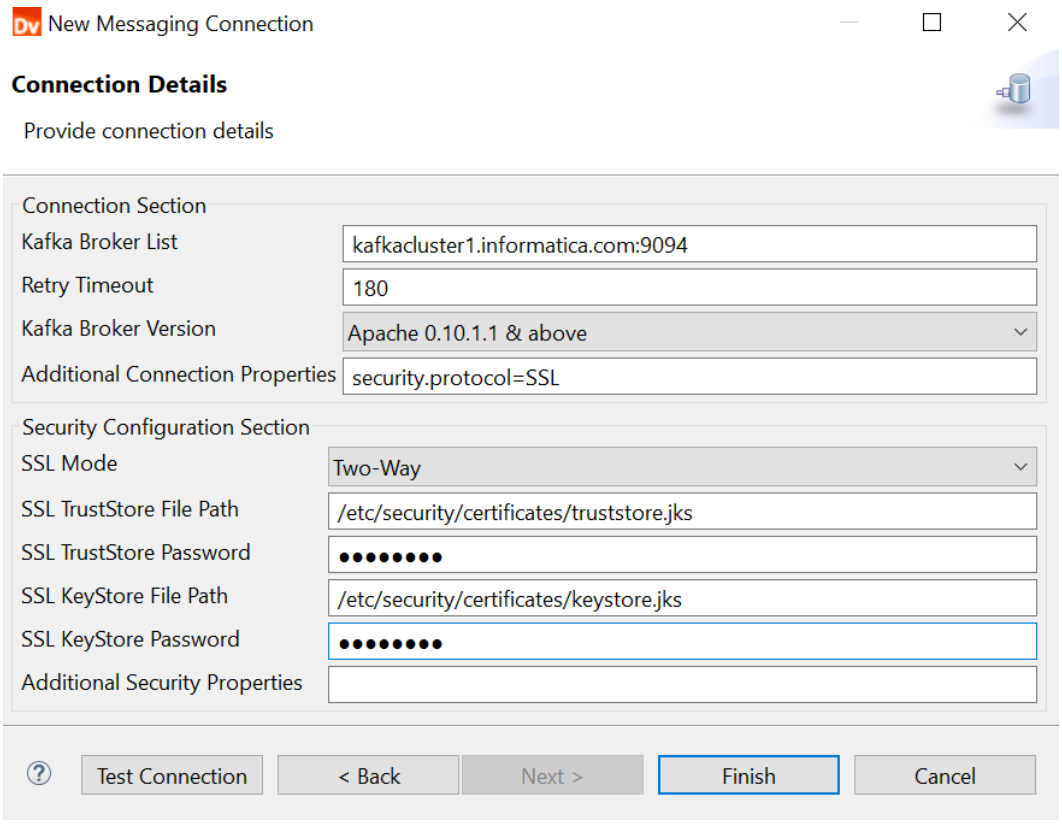
   The following image shows the connection details of a new messaging connection:



2. Configure the additional connection details for the new connection.

Connection Details	Values
Kafka Broker List	`<Fully qualified domain name of Kafka Server>:<Port Number>` Do not type in the IP address of the Kafka broker for SSL Kafka connections.
Additional Connection Properties	`security.protocol=SSL`
SSL Mode	Two-Way.
SSL Truststore FilePath	Truststore path of the local system in Linux format. For example, `/etc/security/certificates/truststore.jks`
SSL Truststore Password	Truststore password.
SSL Keystore Path	Keystore path of the local system in Linux format. For example, `etc/security/certificates/Keyststore.jks`
SSL Keystore Password	Keystore password.

   The following image shows the additional connection details of a new messaging connection:



3. Test the connection and then click **Finish**.

   A connection successful message appears.
   The success message indicates that you have successfully configured an SSL secured Kafka connection.

Authors

Rajat Sharda

Pubali Dekaphukan